CSA4707-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 5: Bias–Variance, Overfitting and Underfitting Analysis

Problem Statement: A real-estate start-up predicts house prices from floor area. One analyst's model is too simple; another memorises the training data. 150 noisy samples of house area versus price, with a hidden non-linear relationship. The objective is to demonstrate underfitting, good fit and overfitting, and estimate bias and variance. Use Scikit-learn, NumPy, Matplotlib.

Tasks:

1. Fit polynomial regression of degree 1, 3, 9 and 15 with Scikit-learn.
2. Plot each fitted curve over the data.
3. Plot training and validation error against model complexity.
4. Estimate bias2 and variance through repeated resampling.
5. Select the best degree and justify it.

FORMULA:

1.Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

2.Bias squared

$$Bias^2 = (E[\hat{y}] - y_{true})^2$$

3.Variance

$$Variance = E[(\hat{y} - E[\hat{y}])^2]$$

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

np.random.seed(10)

X = np.linspace(500, 3000, 150).reshape(-1, 1)
y = 50 + 0.08 * X.ravel() + 0.00003 * X.ravel()**2
y = y + np.random.normal(0, 40, 150)

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=10
)

degrees = [1, 3, 9, 15]
train_errors = []
val_errors = []

plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color="blue", label="Training Data")
plt.scatter(X_val, y_val, color="orange", label="Validation Data")

x_plot = np.linspace(500, 3000, 200).reshape(-1, 1)

for d in degrees:
    model = make_pipeline(PolynomialFeatures(d), LinearRegression())
```

```
    model.fit(X_train, y_train)

    train_errors.append(mean_squared_error(y_train, model.predict(X_train)))
    val_errors.append(mean_squared_error(y_val, model.predict(X_val)))

    plt.plot(x_plot, model.predict(x_plot), label=f"Degree {d}")

plt.xlabel("House Area (sq. ft.)")
plt.ylabel("House Price")
plt.title("Polynomial Regression Fits")
plt.legend()
plt.grid()
plt.show()

plt.plot(degrees, train_errors, "o-", label="Training Error")
plt.plot(degrees, val_errors, "s-", label="Validation Error")
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Training and Validation Errors")
plt.legend()
plt.grid()
plt.show()

predictions = []

for seed in range(30):
    rng = np.random.RandomState(seed)
    indices = rng.choice(len(X), len(X), replace=True)
    model = make_pipeline(PolynomialFeatures(3), LinearRegression())
    model.fit(X[indices], y[indices])
    predictions.append(model.predict(x_plot).ravel())
```

```
predictions = np.array(predictions)
mean_prediction = predictions.mean(axis=0)

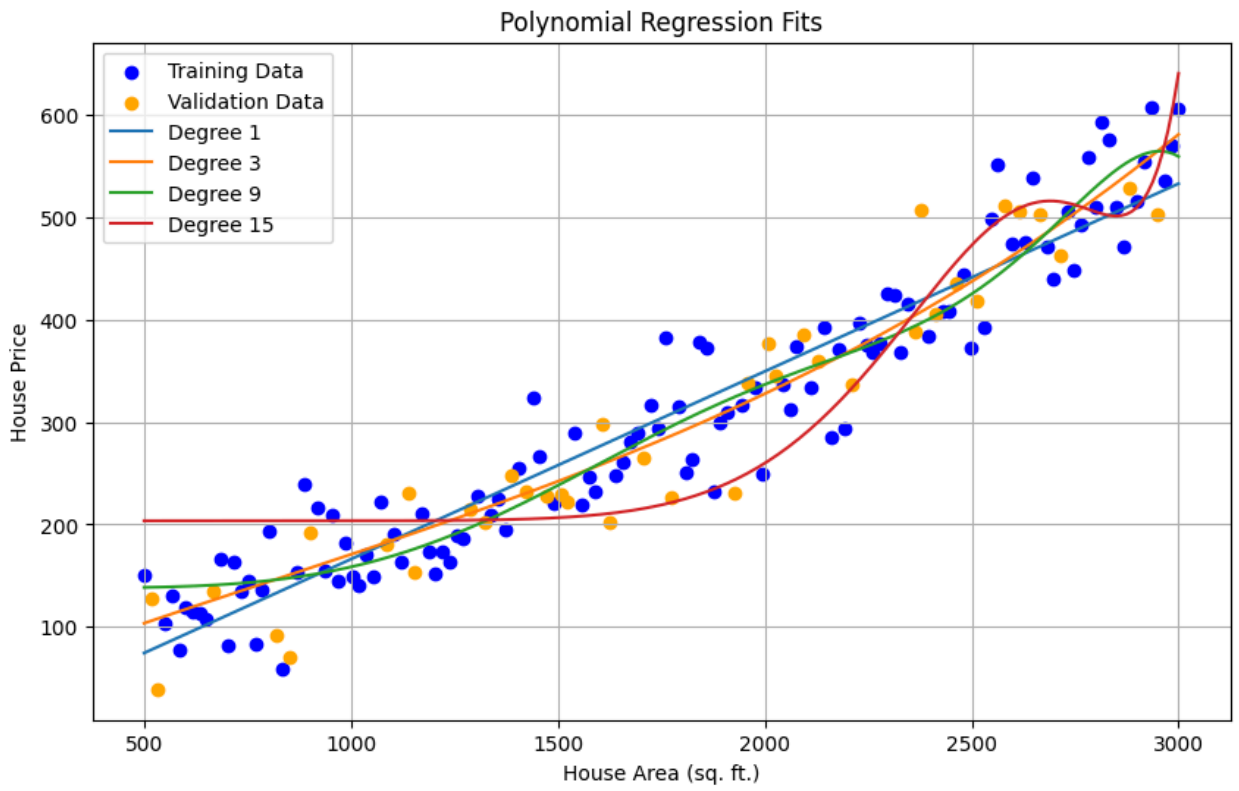
bias_squared = np.mean((mean_prediction - (
    50 + 0.08 * x_plot.ravel()
    + 0.00003 * x_plot.ravel()**2
))**2)

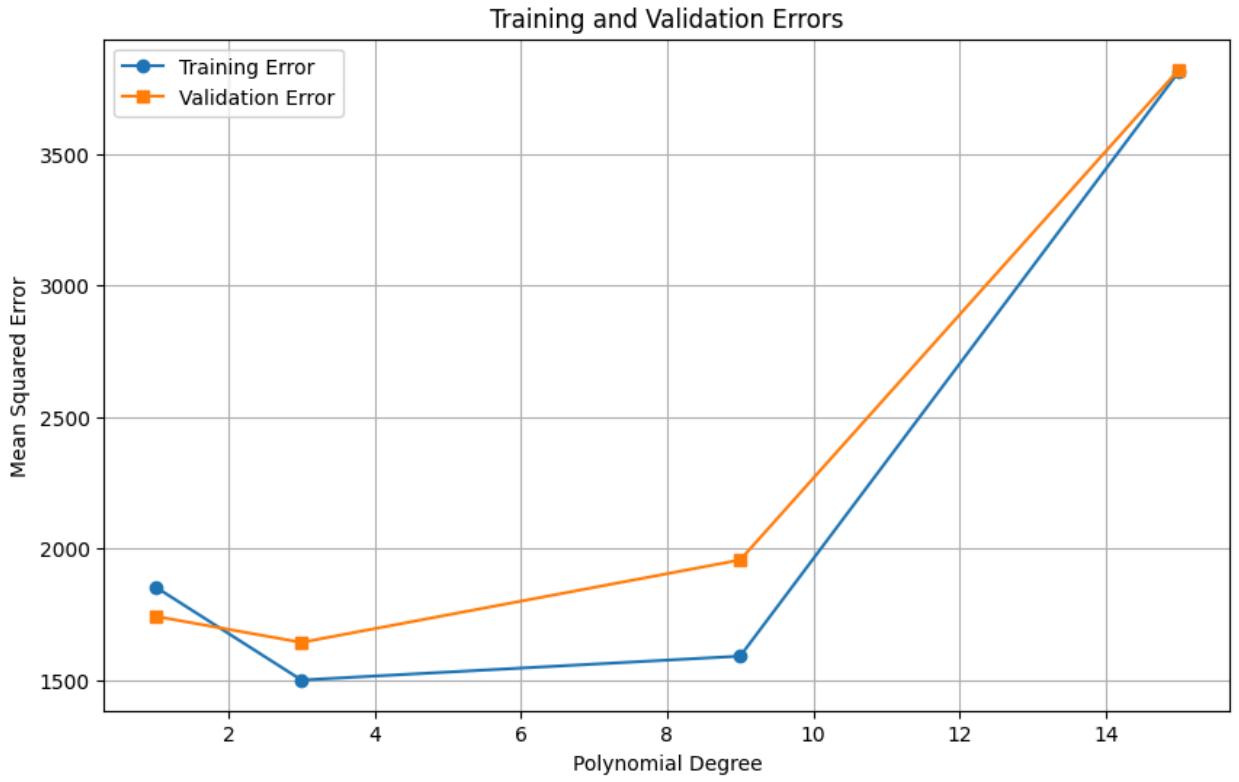
variance = np.mean(np.var(predictions, axis=0))

print("Estimated Bias Squared:", round(bias_squared, 2))
print("Estimated Variance:", round(variance, 2))

best_degree = degrees[np.argmin(val_errors)]
print("Best Polynomial Degree:", best_degree)
```

OUTPUT:





Estimated Bias Squared: 32.67

Estimated Variance: 43.4

Best Polynomial Degree: 3